

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL1- INDUSTRY PROBLEM SOLVING TASK
Weightage:	30%	Total Marks:	25
CO4 Statement:	Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.		
Student Name:	SANTHOSH K	Reg.No.:	192312450
Department:	ECE	Date:	26.8.26

ANNEXURE A
INDUSTRY PROBLEM SOLVING TASK

1. Smartphone Performance Optimization
A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.
2. Cloud Server Memory Bottleneck
A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.
3. Autonomous Vehicle Data Processing
An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.
4. AI Inference Edge Device
An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.
5. High-Performance Gaming Console
A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

Code of Conduct:

I Santhosh K / (192312450) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Santhosh K
Signature of the student:

MARKS ALLOCATION

	Understanding of Industry Problem (20%)	Application of Engineering Concepts (25%)	Solution Design (25%)	Use of Tools (15%)	Analysis (10%)	
Q1	1	1.25	1	0.75	0.5	4.5
Q2	0.8	1.25	1	0.5	0.5	3.75
Q3	0.5	1.25	1	0.75	0.5	4
Q4	1	1.25	1	0.5	0.5	4.25
Q5	1	1	1	0.5	0.5	4

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints(1)	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem(1.25)	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards(1.25)	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data(0.75)	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation(0.5)	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

1. Smartphone Performance Optimization

A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

1. Understanding of Industry Problem

The mobile manufacturer observes lag when users switch between applications. This problem is directly related to the movement of instructions and data through the memory hierarchy. Application switching may require the processor to access frequently used code, user data, operating-system data, graphics buffers, and application state. If the required information is not available in the nearest cache, the processor must access lower memory levels. Repeated cache misses increase processor stall time and make the user interface feel slow.

The major engineering constraints are low access latency, sufficient memory capacity, high memory bandwidth, low power consumption, and acceptable chip area. A smartphone cannot simply use very large SRAM everywhere because SRAM is expensive in area and power. Therefore, a hierarchical approach is required.

- Primary performance objective: reduce average memory access latency.
- Secondary objective: increase effective memory bandwidth and reduce CPU waiting time.
- Design constraint: balance speed, capacity, power, cost, and physical area.

2. Application of Engineering Concepts

L1 cache is the smallest and fastest cache and is located closest to the processor core. It normally stores highly active instructions and data. L2 cache is larger than L1 and provides a second level of fast storage. L3 cache is larger again and can be shared by multiple processor cores. DRAM acts as the main memory and provides much larger capacity, but its latency is significantly higher than cache.

The hierarchy works because programs generally demonstrate temporal locality and spatial locality. Temporal locality means recently used data is likely to be used again, while spatial locality means nearby addresses are likely to be accessed. Cache memory exploits these properties.

- L1: immediate working set and critical instructions/data.
- L2: larger private working set and reduced L1 miss penalty.
- L3: shared data and instruction capacity between cores.
- DRAM: large main-memory capacity for applications and operating-system data.

3. Proposed Solution Design

A practical redesigned hierarchy is:

CPU/Register File

→ L1 Instruction/Data Cache

→ L2 Cache

→ Shared L3 Cache

→ High-bandwidth LPDDR DRAM

→ UFS/NAND storage

The manufacturer should first optimize cache size and associativity based on workload measurements. A larger L3 cache can reduce DRAM traffic, while suitable prefetching can bring predictable data into the cache before the processor requests it. High-bandwidth LPDDR DRAM should provide sufficient capacity for several active applications. Storage should mainly be used for persistent application data and loading, not as normal working memory.

The operating system can also improve responsiveness by keeping frequently used application pages resident in DRAM and reducing unnecessary background memory pressure.

- Use cache prefetching carefully to avoid polluting the cache.
- Use efficient cache replacement policies for smartphone workloads.
- Use memory compression where appropriate to reduce DRAM pressure.
- Prioritize foreground applications for memory and CPU resources.

4. Use of Tools

The proposed design should be validated using software profiling and hardware performance counters. Measurements should be taken before and after the redesign. Cache misses, DRAM accesses, memory bandwidth, CPU stalls, application launch time, and application-switch time can be recorded.

- CPU/cache performance counters for L1/L2/L3 misses.
- Linux/Android tracing and profiling tools for memory behavior.
- Memory-bandwidth benchmarks for DRAM throughput.
- Application-level measurement of app-switch latency and responsiveness.

5. Analysis and Validation

The expected improvement is obtained mainly by reducing the number of slow memory accesses. For example, if an application repeatedly accesses the same working set, a larger effective cache can increase the cache-hit ratio.

Fewer DRAM transactions reduce CPU waiting and improve throughput. However, increasing cache size indefinitely is not practical because it increases silicon area, power, and sometimes access time.

The design should therefore be validated using measured cache-hit rate, average memory access time, DRAM bandwidth, CPU utilization, and application-switch latency. The best design is the one that improves real workload performance rather than simply increasing memory capacity.

- Target result: lower cache-miss rate.
- Target result: lower average memory access latency.
- Target result: smoother and faster application switching.
- Trade-off: cache capacity versus area and power.

6. Conclusion

A balanced L1–L2–L3 cache hierarchy with high-bandwidth LPDDR DRAM is the most practical redesign. Cache locality, prefetching, memory management, and workload-aware scheduling should be combined. This directly addresses the observed lag while maintaining the capacity and power requirements of a smartphone.

2. Cloud Server Memory Bottleneck

A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

1. Understanding of Industry Problem

The cloud service provider experiences high latency in database queries. Database applications repeatedly access indexes, records, transaction buffers, metadata, and intermediate results. When these data structures are not available in fast memory, the CPU must access slower memory levels. Under severe memory pressure, virtual-memory paging can cause page faults and storage accesses, which can increase query latency significantly.

The main constraints are predictable response time, high memory capacity, high throughput, low page-fault frequency, and efficient use of server resources.

- Database query latency must be minimized.
- Frequently accessed database data should remain in fast memory.
- Memory pressure and unnecessary paging should be controlled.

2. Application of Engineering Concepts

Cache hierarchy and virtual memory operate at different levels. L1, L2, and L3 caches reduce CPU-to-memory access latency. Virtual memory provides an abstraction that allows processes to use virtual addresses and pages. When required pages are not available in physical memory, the operating system may need to retrieve them from backing storage. Such page faults are much slower than ordinary DRAM accesses.

Therefore, cache hierarchy should be optimized for processor performance, while sufficient DRAM capacity should be provided to minimize paging.

- L1/L2/L3: reduce processor memory-access latency.
- DRAM: primary working memory for database buffers and active processes.
- Virtual memory: provides address-space management and protection.
- NVMe SSD: persistent storage and backing storage, but should not replace sufficient DRAM.

3. Recommended Solution Design

Recommended hierarchy:

CPU

→ L1 Cache

→ L2 Cache

→ L3 Cache

→ High-capacity DDR5 DRAM

→ NVMe SSD

The database server should allocate sufficient DRAM for its buffer pool so frequently accessed tables and indexes remain in memory. Operating-system configuration should minimize swapping for latency-sensitive database services. Database indexes should be designed to reduce unnecessary memory and storage operations.

NUMA-aware allocation can also be considered in multi-socket systems so that processes access memory close to the CPU socket whenever possible.

- Increase DRAM when monitoring shows memory pressure and paging.
- Optimize database buffer-pool size.
- Use high-speed NVMe for persistent data and controlled I/O.
- Use NUMA-aware memory allocation in large servers.

4. Use of Tools

The system should be evaluated using database monitoring tools and operating-system performance counters. Query latency should be correlated with cache misses, page faults, memory utilization, and storage I/O.

- Linux perf for CPU/cache behavior.
- vmstat for memory pressure and paging.
- iostat for storage activity.
- Database performance monitoring for query execution time and buffer usage.

5. Analysis and Validation

If the server shows high page-fault activity, increasing DRAM capacity can provide a larger improvement than increasing CPU cache size. If paging is already low but CPU cache misses are high, workload optimization and cache behavior become more important. The diagnosis must therefore be measurement-driven.

Performance should be compared using average query latency, tail latency, page-fault rate, cache-miss rate, DRAM utilization, storage I/O, and query throughput.

- Lower page-fault rate indicates reduced dependence on backing storage.
- Higher buffer-cache hit rate indicates better database memory utilization.
- Lower query latency and higher queries-per-second indicate improved throughput.

6. Conclusion

The recommended cloud-server design combines an efficient CPU cache hierarchy, sufficient high-capacity DRAM, optimized database buffering, and fast NVMe storage. Virtual-memory paging should be treated as a mechanism for address-space management rather than as a substitute for adequate physical memory.

3. Autonomous Vehicle Data Processing

An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

1. Understanding of Industry Problem

An autonomous vehicle must process data from multiple sensors in real time. Camera, radar, LiDAR, and other sensor streams generate large volumes of data that must be stored, processed, and transferred quickly. Perception, sensor fusion, path planning, and control algorithms are sensitive to latency.

The memory system therefore requires low latency for time-critical operations, high bandwidth for sensor streams, adequate capacity for large datasets, predictable timing, and controlled power consumption.

- Sensor data must be processed within strict real-time constraints.
- Large sensor streams require high memory bandwidth.
- Time-critical intermediate data requires very low latency.

2. Application of Engineering Concepts

SRAM provides very fast access and is suitable for caches, local buffers, and tightly coupled memory. Its major disadvantage is high area cost and lower density. DRAM provides much higher capacity and is appropriate for large sensor buffers, AI models, and intermediate data. MRAM is non-volatile and can retain information without continuous power, making it useful for selected configuration, persistence, and fast-startup functions.

No single memory technology provides the best combination of latency, density, capacity, power, and persistence. A heterogeneous hierarchy is therefore appropriate.

- SRAM: lowest-latency local memory.
- DRAM: large, high-bandwidth working memory.
- MRAM: non-volatile storage for selected frequently needed persistent information.

3. Proposed Memory Hierarchy

Recommended hierarchy:

Real-Time CPU / AI Accelerator

→ SRAM L1 / Local Scratchpad

→ SRAM L2 / Tightly Coupled Memory

→ High-Bandwidth DRAM

→ MRAM for selected persistent/configuration information

→ Vehicle Non-Volatile Storage

Time-critical sensor buffers and AI intermediate values should be placed as close as possible to the processing units. Large frames, point clouds, and model parameters can be held in DRAM. MRAM should be used selectively where fast non-volatility provides a meaningful system benefit.

- Keep control-loop data in fast local SRAM.
- Use DRAM for large sensor and AI workloads.
- Use MRAM selectively for persistent parameters and fast-start information.
- Use storage for long-term logging rather than time-critical processing.

4. Use of Tools

Real-time profiling and hardware performance counters can measure memory stalls and bandwidth usage. Sensor-to-decision latency should be measured under realistic traffic conditions. AI inference benchmarks should also be used to evaluate model execution.

- Real-time software profilers.
- Memory-bandwidth and latency benchmarks.
- Hardware performance counters.
- AI inference and sensor-fusion benchmarks.

5. Analysis and Validation

The design should be evaluated using worst-case latency as well as average latency. Average performance alone is insufficient for safety-related real-time processing because occasional long memory delays can affect system timing.

Important measurements include sensor-to-decision latency, memory bandwidth utilization, SRAM/DRAM access patterns, cache misses, AI inference time, and energy per processing cycle.

- Low worst-case latency improves real-time responsiveness.
- High DRAM bandwidth supports continuous sensor streams.
- SRAM reduces stalls for critical working sets.
- MRAM can reduce dependence on volatile storage for selected persistent data.

6. Conclusion

A hybrid SRAM–DRAM–MRAM architecture provides an effective balance of latency, bandwidth, capacity, persistence, and power. The hierarchy should be designed around the real-time criticality of each data type rather than treating all sensor data equally.

4. AI Inference Edge Device

An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

1. Understanding of Industry Problem

The edge AI device needs fast model loading while operating under limited power and memory constraints. Machine-learning models contain parameters that may be too large for on-chip SRAM. During inference, the

processor repeatedly accesses weights, activations, and intermediate results. The memory architecture must therefore reduce loading time, provide adequate bandwidth, and avoid unnecessary power consumption.

- Fast model loading is required.
- Power consumption must remain low.
- Model weights and activations need efficient high-speed access.

2. Application of Engineering Concepts

NAND Flash provides high-density non-volatile storage and is appropriate for permanent model storage. DRAM provides fast working memory and can hold model parameters and intermediate results during inference. SRAM is extremely fast and is useful for small, frequently accessed data. RRAM is a non-volatile technology that may be used in selected applications where fast access and persistence are valuable.

The hierarchy should separate bulk storage from active inference memory.

- NAND Flash: bulk persistent model storage.
- DRAM: main working memory.
- SRAM: immediate local cache/scratchpad.
- RRAM: potential fast non-volatile memory for selected workloads.

3. Optimized Hierarchical Design

AI Processor

→ On-chip SRAM

→ DRAM

→ RRAM for selected model data

→ NAND Flash

At startup, the required model is transferred from NAND Flash into DRAM. Frequently reused model parameters and activations can be moved into SRAM when their working set is small enough. Model compression and quantization can reduce storage size and memory traffic, provided accuracy remains acceptable.

The system should avoid repeatedly reading the same model data directly from Flash during inference.

- Use SRAM for hot data and immediate computation.
- Use DRAM for the active model and intermediate tensors.
- Use NAND Flash for persistent bulk storage.
- Evaluate RRAM for selected low-power/non-volatile use cases.
- Use compression or quantization to reduce memory traffic where acceptable.

4. Use of Tools

The system can be evaluated using AI inference benchmarks, power measurement instruments, memory profilers, and storage throughput tests. Model-loading time and energy per inference should be measured under representative workloads.

- AI model inference benchmarks.
- Power and energy measurement.
- Memory bandwidth profiling.
- Flash read-throughput and model-loading tests.

5. Analysis and Validation

The design must distinguish model-loading performance from inference performance. NAND Flash affects loading time, whereas DRAM bandwidth and SRAM locality strongly influence active inference. A larger memory capacity is not automatically better if it increases power consumption without reducing the bottleneck.

Important metrics are model loading time, inference latency, energy per inference, DRAM bandwidth utilization, SRAM hit rate, and storage read throughput.

- Lower model-loading time improves startup responsiveness.
- Lower energy per inference improves battery life.
- Higher effective memory bandwidth improves AI throughput.
- Reducing repeated Flash accesses reduces unnecessary power and latency.

6. Conclusion

The optimized design uses NAND Flash for high-capacity persistence, DRAM for active model execution, and SRAM for the most frequently accessed data. RRAM can complement the hierarchy in selected edge-AI designs where non-volatility and fast access justify its use.

5. High-Performance Gaming Console

A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

1. Understanding of Industry Problem

The gaming console experiences frame drops when large scenes are loaded. Modern scenes can contain large textures, geometry, shaders, audio, and world-state information. These assets must move from persistent storage to working memory and finally to CPU/GPU processing resources.

If the memory system cannot deliver data quickly enough, the processor or graphics engine may wait for data. This increases frame time and causes visible stuttering or frame drops.

- Large scene assets create high memory traffic.
- Frame rendering requires predictable and continuous data delivery.
- Both memory bandwidth and storage streaming can become bottlenecks.

2. Application of Engineering Concepts

L1 and L2 caches provide very fast access to processor-local working sets. L3 cache offers a larger shared cache and can reduce DRAM accesses when data has sufficient temporal locality. DRAM provides the large working capacity required for active game assets. High-speed SSD storage supplies assets that cannot fit in DRAM.

L3 cache should not be considered a replacement for DRAM. Instead, the two should work together: cache handles high-locality data while DRAM handles the larger active dataset.

- L1/L2: immediate CPU working data.
- L3: larger shared cache and reduction of main-memory traffic.
- High-bandwidth DRAM: active game assets and large working sets.
- SSD: persistent game installation and streaming source.

3. Proposed Solution Design

CPU/GPU

→ L1 Cache

→ L2 Cache

→ Large Shared L3 Cache

→ High-Bandwidth GDDR/DRAM

→ High-Speed SSD

The console should use efficient asset streaming so that required data is transferred before it is needed. Prefetching can hide storage latency. Asset compression can reduce the amount of data transferred. Cache policies should prioritize frequently reused game data rather than allowing one large streaming operation to evict useful working data.

The graphics pipeline should also be evaluated so that memory bandwidth is balanced between CPU and GPU requirements.

- Optimize L3 cache capacity and replacement policy.
- Use high-bandwidth graphics-oriented DRAM.

- Stream assets asynchronously from SSD.
- Use prefetching to reduce visible loading delays.
- Use asset compression to reduce transfer volume.

4. Use of Tools

Game-engine profiling and hardware performance counters can identify whether frame drops are caused by cache misses, DRAM bandwidth saturation, storage latency, CPU stalls, or GPU memory traffic. Frame-time analysis should be performed during the exact scene that produces the problem.

- CPU/GPU performance counters.
- Frame-time and 1% low-FPS analysis.
- Memory-bandwidth benchmarks.
- SSD throughput and asset-streaming measurements.
- Game-engine profiling tools.

5. Analysis and Validation

The system should measure frame time rather than relying only on average FPS. A game may show a high average FPS while still producing visible stutters due to occasional long frames. Therefore, 1% low FPS, frame-time spikes, cache misses, DRAM bandwidth utilization, and asset-loading time should all be evaluated.

If cache misses dominate, improving cache locality may help. If DRAM bandwidth is saturated, increasing memory bandwidth or reducing data movement is more appropriate. If SSD streaming is the bottleneck, asynchronous loading, compression, and faster storage can help.

- Lower frame-time spikes indicate smoother rendering.
- Higher effective DRAM bandwidth supports continuous rendering.
- Lower cache-miss rates reduce processor stalls.
- Lower asset-loading time reduces scene-transition and streaming delays.

6. Conclusion

A coordinated L1–L2–L3 cache hierarchy, high-bandwidth DRAM, and fast SSD streaming provides a practical solution to frame drops. The final design should be selected from measured bottleneck data rather than simply increasing one memory component. This approach directly connects cache hierarchy, bandwidth, throughput, and real-world gaming performance.